NLP Labs & Dimensionality Reduction: PCA, t-SNE, UMAP

KAUST–Mawhiba IOAI Training

Week 3 · Day 14

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

May 5, 2026

Table of Contents

1. Recap & Motivation
2. Why Dimensionality Reduction?
3. Principal Component Analysis (PCA)
4. t-SNE: Nonlinear Dimensionality Reduction
5. UMAP: Fast Nonlinear Reduction
6. Comparison & Summary

Recap: Days 12–13 in One Slide

- **Tokenization:** splitting text into units (words, sub-words via BPE)
- **Bag of Words & TF-IDF:** sparse document vectors
- **Word Embeddings:** dense vectors learned from data
  - Word2Vec, GloVe, fastText
  - Typically **100–300 dimensions** per word
- **Similarity:** cosine similarity measures semantic closeness
- **Word Arithmetic:** $\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$

Where We Left Off

We can now represent every word as a vector in $\mathbb{R}^{300}$:

	$\mathbf{d}_1$	$\mathbf{d}_2$	$\mathbf{d}_3$	$\mathbf{d}_4$	$\cdots$	$\mathbf{d}_{300}$
king	0.52	-0.31	0.18	0.94	$\cdots$	-0.07
queen	0.48	-0.27	0.22	0.89	$\cdots$	-0.05
cat	-0.61	0.73	-0.15	0.03	$\cdots$	0.44
dog	-0.55	0.68	-0.11	0.08	$\cdots$	0.39

Problem: We know "king" and "queen" are close, but...

How do you *visualize* 300-dimensional data?

Today's Roadmap

Morning: NLP Hands-On Labs

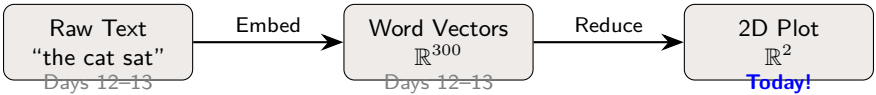
1. Tokenization lab — implement BPE, use HuggingFace tokenizers
2. Word embeddings lab — word analogies, cosine similarity, text search

Afternoon: Dimensionality Reduction

1. **PCA** — linear projection, find maximum variance directions
2. **t-SNE** — nonlinear, preserve neighborhoods
3. **UMAP** — fast nonlinear, preserve global structure

Goal: Go from 300D word vectors → a 2D scatter plot where similar words cluster together.

The Big Picture: From Words to Pictures



Why does this matter?

- **Visualization:** see clusters, outliers, relationships
- **Preprocessing:** reduce noise before classification
- **Speed:** fewer dimensions = faster training
- **Insight:** discover hidden structure in your data

Why **Dimensionality Reduction**?

What Is a "Dimension"?

Each **feature** in your data is one dimension:

Data Type	Features	Dimensions
Tabular (diabetes)	age, BMI, glucose, ...	$\sim$10
Image (28$\times$28 grayscale)	pixel intensities	784
Word embedding (GloVe)	learned dimensions	300
Document (BoW, 50K vocab)	word counts	50,000

High-dimensional data is everywhere in ML.

Humans can only see **2D** (or 3D) — we need to *compress* dimensions while keeping the important structure.

The Curse of Dimensionality

As dimensions increase, strange things happen:

- **Distances converge:** in high-D, the nearest and farthest neighbors are almost the same distance apart

$$\frac{d_{\max} - d_{\min}}{d_{\min}} \to 0 \quad \text{as } D \to \infty$$

- **Data becomes sparse:** 10 points in 1D $\to$ dense coverage; 10 points in 100D $\to$ nearly empty space

- **Overfitting risk:** more features than samples $\to$ model memorizes noise

Intuition

Imagine searching for your friend in a 1D hallway vs. a 100D hypercube. In high dimensions, *everywhere is far from everywhere else*.

Goals of Dimensionality Reduction

Reduce D features $\rightarrow$ d features ($d \ll D$), while:

1. **Preserving structure:**
   - Points that are close in high-D should stay close in low-D
   - Clusters should remain visible
2. **Removing noise:**
   - Many dimensions may carry redundant or noisy information
   - Keeping only the "important" directions cleans the signal
3. **Enabling visualization:**
   - Project to 2D or 3D for human inspection
   - Discover patterns, clusters, outliers

Two Families of Methods

Linear Methods

- New coordinates are **linear combinations** of original features
- Fast, deterministic, interpretable
- **Example:** PCA

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_D x_D$$

Nonlinear Methods

- Learn **curved** manifolds in the data
- Better at capturing complex structure
- **Examples:** t-SNE, UMAP

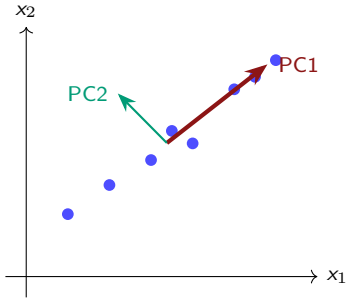
$$z = f(x_1, x_2, \ldots, x_D) \quad \text{(nonlinear } f\text{)}$$

Today we learn all three: PCA $\rightarrow$ t-SNE $\rightarrow$ UMAP

Principal Component Analysis: **PCA**

PCA: The Core Idea

Find the directions where data varies the most, then project onto those directions.



- **PC1:** direction of *maximum variance*
- **PC2:** perpendicular to PC1, captures remaining variance
- Projecting onto PC1 alone gives the best 1D summary

Analogy

PCA is like finding the "spine" of a data cloud — the line (or plane) along which the data is most spread out.

PCA: The Algorithm (Overview)

Given n data points in D dimensions, reduce to d dimensions ($d < D$):

1. **Center** the data: subtract the mean of each feature
2. **Compute** the covariance matrix $C \in \mathbb{R}^{D \times D}$
3. **Find** eigenvectors and eigenvalues of C
4. **Sort** eigenvectors by eigenvalue (largest first)
5. **Project** data onto the top d eigenvectors

Result: each data point is now a vector in $\mathbb{R}^d$.

Let's walk through each step with a concrete example!

Our Running Example: 5 Students, 2 Subjects

Dataset: 5 students, each with a Math and Science score.

Student	**Math** (x_1)	**Science** (x_2)
Alice	90	85
Bob	70	70
Carol	50	55
Dave	80	75
Eve	60	65

Question: Can we summarize each student with a *single number* instead of two? (2D $\rightarrow$ 1D)

We will do this step-by-step, then show Python does it in 3 lines.

Step 1: Center the Data

Subtract the mean of each feature so the data is centered at the origin.

Mean: $\bar{x}_1 = \frac{90+70+50+80+60}{5} = 70, \quad \bar{x}_2 = \frac{85+70+55+75+65}{5} = 70$

	Math	Science	Centered x_1	Centered x_2
Alice	90	85	$+20$	$+15$
Bob	70	70	0	0
Carol	50	55	-20	-15
Dave	80	75	$+10$	$+5$
Eve	60	65	-10	-5

Why center? PCA finds directions of variance *from the mean*. Without centering, the first component would just point toward the mean itself.

Step 2: Compute the Covariance Matrix

The **covariance matrix** measures how features vary *together*:

$$C = \frac{1}{n-1} X_c^T X_c \qquad \text{where } X_c \text{ is the centered data matrix}$$

$$X_c = \begin{pmatrix} 20 & 15 \\ 0 & 0 \\ -20 & -15 \\ 10 & 5 \\ -10 & -5 \end{pmatrix}$$

$$C = \tfrac{1}{4} \begin{pmatrix} 1000 & 700 \\ 700 & 500 \end{pmatrix} = \begin{pmatrix} 250 & 175 \\ 175 & 125 \end{pmatrix}$$

Reading the matrix:

- **Diagonal** $(250, 125)$: variance of Math, variance of Science
- **Off-diagonal** (175): covariance — Math and Science move together
- Positive covariance $\rightarrow$ high Math $\Leftrightarrow$ high Science

Quick Review: Variance and Covariance

- **Variance** of feature x:

$$\text{Var}(x) = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

  Measures how spread out a single feature is.

- **Covariance** of features x and y:

$$\text{Cov}(x, y) = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})$$

  Measures how two features change *together*.

- $\text{Cov} > 0$: both increase together $\text{Cov} < 0$: one increases as the other decreases

Our covariance $175 > 0$ confirms: students good at Math tend to be good at Science.

Step 3: Find Eigenvectors and Eigenvalues

We solve $C\mathbf{v} = \lambda\mathbf{v}$ to find:

- **Eigenvectors** $\mathbf{v}$: the *directions* of maximum variance
- **Eigenvalues** λ: the *amount* of variance in each direction

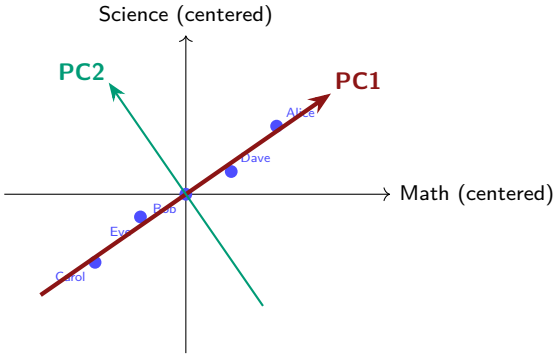
For our covariance matrix $C = \begin{pmatrix} 250 & 175 \\ 175 & 125 \end{pmatrix}$:

	Eigenvalue λ	**Eigenvector** $\mathbf{v}$
PC1	$\lambda_1 = 366.3$	$\mathbf{v}_1 = (0.82,\ 0.57)$
PC2	$\lambda_2 = 8.7$	$\mathbf{v}_2 = (-0.57,\ 0.82)$

Key observations:

- $\lambda_1 \gg \lambda_2$ — almost all variance is along PC1
- PC1 direction $(0.82, 0.57)$ is a mix of *both* subjects $\rightarrow$ "overall ability"

What Do Eigenvectors Look Like?



- **PC1** passes through the "spine" of the data cloud
- **PC2** is perpendicular — captures what PC1 misses
- Data is much more spread along PC1 than PC2

Step 4: Explained Variance Ratio

How much information does each component capture?

$$\text{Explained Variance Ratio of PC}_k = \frac{\lambda_k}{\sum_{j=1}^{D} \lambda_j}$$

	λ	**Variance %**	**Cumulative %**
PC1	366.3	$\frac{366.3}{375.0} = 97.7\%$	97.7%
PC2	8.7	$\frac{8.7}{375.0} = 2.3\%$	100%

PC1 captures 97.7% of all variance!

This means: we can compress 2 features into 1 and lose only 2.3% of the information.

In practice: keep enough PCs to capture $\geq 90\%$ of variance.

Multiply each centered data point by the PC1 eigenvector:

$$z_i = \mathbf{v}_1^T \cdot \mathbf{x}_{c,i} = 0.82 \cdot x_{c,1} + 0.57 \cdot x_{c,2}$$

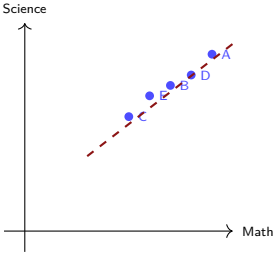
	Centered Math	Centered Sci	PC1 score z
Alice	$+20$	$+15$	$0.82(20) + 0.57(15) = \mathbf{+24.95}$
Bob	0	0	$0.82(0) + 0.57(0) = \mathbf{0.00}$
Carol	-20	-15	$0.82(-20) + 0.57(-15) = \mathbf{-24.95}$
Dave	$+10$	$+5$	$0.82(10) + 0.57(5) = \mathbf{+11.05}$
Eve	-10	-5	$0.82(-10) + 0.57(-5) = \mathbf{-11.05}$

The PC1 score is a single number that ranks students by "overall ability":

Carol (-25.0) Eve (-11.1) Bob (0) Dave $(+11.1)$ Alice $(+25.0)$

Visualizing the Projection

Original 2D data:



Projected to 1D (PC1):

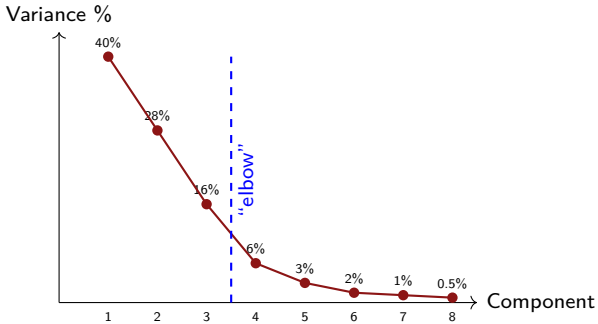
- Left: 2D scatter plot (2 numbers per student)
- Right: 1D projection (1 number per student)
- The **relative order and spacing** are preserved

This is exactly what PCA does with 300D word embeddings → 2D!

Choosing the Number of Components: The Scree Plot

In practice, you don't reduce from 2D to 1D — you reduce from **300D to 2D, 10D, 50D, …**

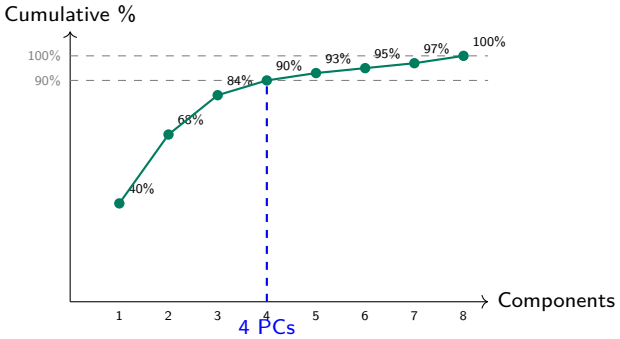
Scree plot: plot eigenvalues (or explained variance %) vs. component number.



Rule of thumb: keep components before the "elbow" where variance drops off

Cumulative Variance: The 90% Rule

An alternative view: plot **cumulative** explained variance.



Common choices:

- **Visualization:** $d = 2$ or $d = 3$ (always)
- **Preprocessing:** keep enough to reach 90–95% variance

PCA on Word Embeddings

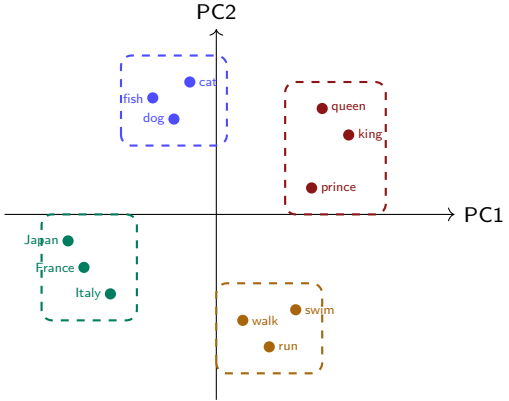
Real application: take 300D GloVe vectors $\rightarrow$ project to 2D for visualization.

1. Collect vectors for words of interest (e.g., animals, countries, verbs)
2. Center the data (subtract mean vector)
3. Compute the 300×300 covariance matrix
4. Find top 2 eigenvectors
5. Project each word vector to 2D
6. Plot and color by category!

What to expect

After PCA to 2D, you should see animals clustered together, countries

Example: PCA on 12 Word Vectors



PCA reveals semantic clusters! Royalty, Animals, Countries, Verbs group together.

PCA in Python: 3 Lines with Scikit-Learn

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_2d = pca.fit_transform(X_300d)

# X_300d: shape (n_words, 300)
# X_2d:   shape (n_words, 2)  -- ready to plot!
```

Useful attributes after fitting:

- `pca.explained_variance_ratio_` — how much variance each PC captures
- `pca.components_` — the eigenvectors (directions)
- `pca.singular_values_` — related to eigenvalues

PCA in Python: Full Example

```python
import numpy as np
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Example: 5 students, 2 subjects
X = np.array([[90,85],[70,70],[50,55],[80,75],[60,65]])
names = ['Alice','Bob','Carol','Dave','Eve']

# Fit PCA (2D -> 1D)
pca = PCA(n_components=1)
X_1d = pca.fit_transform(X)

print(f"Explained variance: "
      f"{pca.explained_variance_ratio_[0]:.1%}")
# Output: Explained variance: 97.7%

# Scree plot for higher-D data:
pca_full = PCA().fit(X_300d)
plt.plot(np.cumsum(pca_full.explained_variance_ratio_))
plt.xlabel('Number of Components')
plt.ylabel('Cumulative Variance Explained')
plt.axhline(y=0.9, color='r', linestyle='--')
plt.show()
```

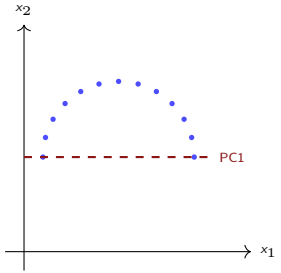
PCA: Strengths

- **Fast:** closed-form solution (eigen-decomposition), scales to large D
- **Deterministic:** same input $\rightarrow$ same output (no randomness)
- **Interpretable:** each PC is a weighted sum of original features
- **Preserves global structure:** distances and variances are maintained
- **Great for preprocessing:** reduce features before training a classifier
- **Explained variance:** tells you exactly how much information you keep

PCA: Limitations

- **Linear only:** can't capture curved or twisted structures
- **Assumes variance = importance:** high variance direction might be noise
- **Visualization limited:** 2 PCs may capture too little variance for complex data

When PCA fails:
If data lies on a curve (manifold), PCA projects onto a straight line and merges clusters.



Data on a curve → PCA projection loses the structure.

This is exactly why we need **nonlinear methods** like t-SNE and UMAP!

PCA: Key Takeaways

1. **PCA finds orthogonal directions of maximum variance**
2. The algorithm: center $\rightarrow$ covariance $\rightarrow$ eigenvectors $\rightarrow$ project
3. Use the **scree plot** or **90% rule** to choose how many components
4. **Strengths:** fast, deterministic, interpretable, good for preprocessing
5. **Weakness:** linear — misses curved/nonlinear structure
6. In Python: `PCA(n_components=k).fit_transform(X)`

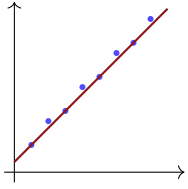
Up next: t-SNE — a nonlinear method that preserves *local neighborhoods*.

t-SNE: **Nonlinear Dimensionality Reduction**

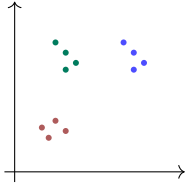
Why Not Just Use PCA?

PCA is great when structure is **linear**, but real data often isn't.

PCA works well:



PCA fails:



With clusters in high-D:

- PCA might project two separate clusters onto the *same* region
- We need a method that preserves **which points are neighbors**

t-SNE: The Core Idea

t-SNE = **t**-distributed **S**tochastic **N**eighbor **E**mbedding

- **Goal:** if two points are close in high-D, keep them close in low-D
- **Approach:**
  1. Measure how "similar" every pair of points is in high-D
  2. Place points randomly in 2D
  3. Iteratively move points in 2D until neighbors match the high-D layout
- **Result:** a 2D map where *neighborhoods are preserved*

Analogy

Imagine you have a social network with 1000 people. You want to place them on a map so that friends are close together. That's what t-SNE does with data points.

How t-SNE Works (Intuition)

Step 1: Measure similarities in high-D

For every pair of points, compute a **similarity score**:

- Close points → **high** similarity
- Far points → **low** similarity (near zero)

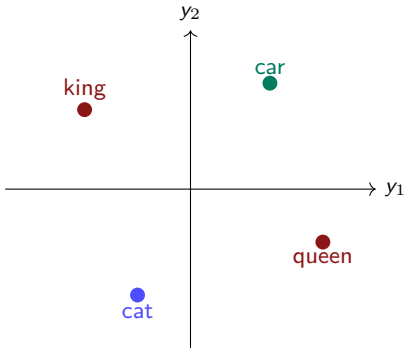
Example with 4 word vectors:

Similarity	king	queen	cat	car
king	—	**high**	low	low
queen	**high**	—	low	low
cat	low	low	—	low
car	low	low	low	—

King and queen are close in 300D → high similarity score.

Cat and car are far from everything → low similarity scores.

Step 2: Start with random positions in 2D



Random positions — king and queen are far apart!

The initial placement is random and wrong.

t-SNE will **iteratively rearrange** points to fix this.

How t-SNE Works (Intuition)

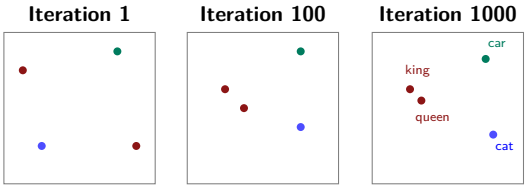
Step 3: Iteratively move points to match the similarities

- Points that should be close (high similarity) are **attracted**
- Points that should be far (low similarity) are **repelled**
- Repeat for hundreds of iterations until the layout stabilizes

Think of it like a **physics simulation**:

- Similar points have *springs* pulling them together
- Dissimilar points have *repulsive forces* pushing them apart
- The system settles into an equilibrium

Watching t-SNE Converge



- King and queen are pulled **together** (high similarity in 300D)
- Cat and car stay **apart** (low similarity in 300D)
- The exact positions don't matter — only **relative closeness**

The Perplexity Parameter

Perplexity controls how many neighbors each point pays attention to.

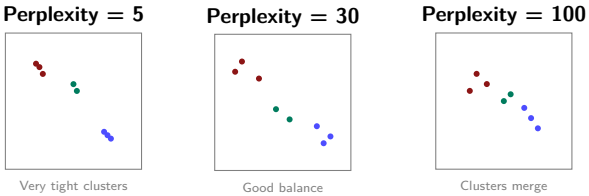
Think of it as: "How many close friends does each point have?"

- **Low perplexity** (5–10): each point only considers its *nearest few* neighbors $\rightarrow$ very tight, small clusters
- **Medium perplexity** (30–50): a good balance between local detail and overall shape
- **High perplexity** (100+): each point considers *many* neighbors $\rightarrow$ clusters may merge

Rule of Thumb

Typical range: **5–50**. Default in scikit-learn: **30**. Try a few values and compare the plots.

Effect of Perplexity: Same Data, Different Views



- Too low: fragmented, spurious sub-clusters
- Too high: clusters blend together, lose local detail
- **Always try multiple values** and look at the results!

t-SNE in Python

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

tsne = TSNE(n_components=2, perplexity=30,
            random_state=42)
X_2d = tsne.fit_transform(X_300d)

plt.scatter(X_2d[:, 0], X_2d[:, 1],
            c=categories, cmap='tab10')
plt.title('t-SNE of Word Embeddings')
plt.show()
```

Key parameters:

- `perplexity` = 30 (default, try 5–50)
- `n_iter` = 1000 (default, increase if not converged)
- `random_state` = 42 (for reproducibility)

t-SNE: Important Caveats

1. **Non-deterministic:** different random seeds → different plots. Always set `random_state`!
2. **Distances between clusters are meaningless:** only the points *inside* each cluster are reliable
3. **Axes are meaningless:** no "PC1 explains 40% variance"
4. **Cannot add new points:** to embed a new point, you must rerun on the entire dataset
5. **Cluster sizes can be misleading:** t-SNE may inflate small clusters and compress large ones
6. **Slow for large datasets:** works well up to ∼50K points

t-SNE is for **visualization only**, not as input to another model.

What t-SNE Preserves (and Doesn't)

Property	Preserved?	Reliable?
Points in the same cluster	✓	Yes
Local neighborhood (nearest neighbors)	✓	Yes
Distance between clusters	✗	No
Cluster sizes / shapes	✗	No
Global layout / orientation	✗	No
Explained variance	✗	N/A

Bottom line: if two points are close in a t-SNE plot, they are likely similar. If two *clusters* are far apart, that may or may not be meaningful.

t-SNE: Key Takeaways

1. **t-SNE preserves local neighborhoods** — similar points stay close
2. Works by iteratively moving 2D points until they match high-D similarities
3. **Perplexity** controls how many neighbors to consider (try 5–50)
4. **Great for:** visualizing clusters in word embeddings, images, etc.
5. **Limitations:** slow, non-deterministic, distances between clusters are meaningless
6. In Python: `TSNE(perplexity=30).fit_transform(X)`

Up next: UMAP — faster, preserves more global structure.

UMAP: **Fast Nonlinear Reduction**

t-SNE Is Great, But...

- **Slow:** $O(n^2)$ — minutes for 10K points, hours for 100K+
- **No global structure:** distances between clusters are meaningless
- **Can't add new points:** must rerun on entire dataset
- **Non-deterministic:** hard to reproduce results

UMAP (Uniform Manifold Approximation and Projection) addresses all of these.

McInnes, Healy & Melville (2018) — now the most popular visualization method.

UMAP: The Core Idea

Like t-SNE, UMAP preserves neighborhoods. But the math is different:

1. **Build a graph** in high-D:
   - Connect each point to its k nearest neighbors
   - Edge weights = distance-based similarity
2. **Optimize a layout** in 2D:
   - Find a 2D arrangement where the graph structure is preserved
   - Uses cross-entropy instead of KL divergence

Key Difference from t-SNE

t-SNE considers *all* pairwise distances ($O(n^2)$). UMAP only considers *nearest neighbors* ($O(n \log n)$) — much faster!

UMAP's Two Key Parameters

`n_neighbors` (default: 15)

How many neighbors to consider for each point.

- **Small** (5): very local, tight clusters
- **Medium** (15): balanced view
- **Large** (50+): broader, more global structure

Similar to t-SNE's perplexity.

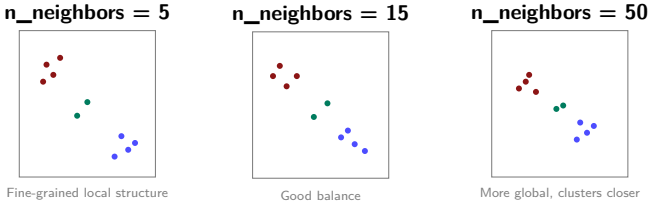
`min_dist` (default: 0.1)

Minimum distance between points in 2D.

- **Small** (0.0): points can overlap, very tight clusters
- **Medium** (0.1): slight spacing
- **Large** (0.5+): spread out, more uniform

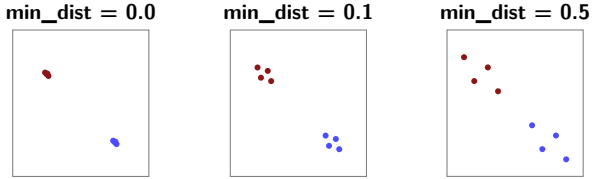
No equivalent in t-SNE.

Effect of n_neighbors



- Small `n_neighbors`: emphasizes local clusters (may fragment)
- Large `n_neighbors`: preserves global relationships (may merge)

Effect of `min_dist`



- `min_dist = 0.0`: tight clusters, good for cluster identification
- `min_dist = 0.5`: spread out, better for seeing internal structure

UMAP vs. t-SNE: Speed Comparison

Dataset Size	t-SNE	UMAP
1,000 points	$\sim$5 sec	$\sim$2 sec
10,000 points	$\sim$2 min	$\sim$15 sec
100,000 points	$\sim$30 min	$\sim$2 min
1,000,000 points	hours / infeasible	$\sim$15 min

Why is UMAP faster?

- t-SNE: computes *all* n^2 pairwise distances
- UMAP: only computes *k-nearest neighbor* distances ($n \log n$)
- UMAP uses stochastic gradient descent with negative sampling

UMAP Preserves More Global Structure

t-SNE:

- Local neighborhoods: ✓
- Cluster separations: ✗
- Relative cluster positions: ✗

Clusters may appear in arbitrary positions. The distance *between* clusters is meaningless.

UMAP:

- Local neighborhoods: ✓
- Cluster separations: ✓ (partially)
- Relative cluster positions: ✓ (partially)

Clusters that are related in high-D tend to be *closer* in the UMAP plot.

Example

In a UMAP of word embeddings: "king" and "queen" clusters would be closer to each other than to "cat" and "dog" clusters — t-SNE doesn't guarantee this.

UMAP in Python

```
# pip install umap-learn
import umap
import matplotlib.pyplot as plt

reducer = umap.UMAP(n_neighbors=15,
                    min_dist=0.1,
                    random_state=42)
X_2d = reducer.fit_transform(X_300d)

plt.scatter(X_2d[:, 0], X_2d[:, 1],
            c=categories, cmap='tab10', s=10)
plt.title('UMAP of Word Embeddings')
plt.show()
```

Key parameters:

- `n_neighbors` = 15 (default, try 5–50)
- `min_dist` = 0.1 (default, try 0.0–0.5)

UMAP: Bonus Features

1. **Transform new points:**
   - After fitting, call `reducer.transform(X_new)`
   - t-SNE cannot do this — must rerun on everything
2. **Supervised UMAP:**
   - Pass labels: `reducer.fit(X, y=labels)`
   - Uses label information to improve cluster separation
3. **Higher-dimensional output:**
   - Not just 2D — can project to 10D, 50D for downstream ML
   - Use as a preprocessing step (like PCA, but nonlinear)

UMAP is both a **visualization tool** and a **feature extraction method**.

UMAP: Key Takeaways

1. **UMAP builds a nearest-neighbor graph** and optimizes a 2D layout
2. Two main parameters: `n_neighbors` (locality) and `min_dist` (density)
3. **Much faster** than t-SNE ($O(n \log n)$ vs. $O(n^2)$)
4. **Preserves more global structure** than t-SNE
5. **Can transform new data** without refitting
6. Install via `pip install umap-learn`

PCA vs. t-SNE vs. UMAP

Head-to-Head Comparison

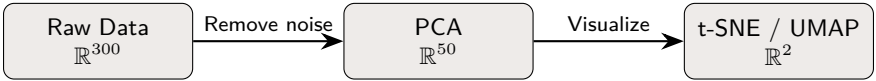
	PCA	t-SNE	UMAP
Type	Linear	Nonlinear	Nonlinear
Speed	Very fast	Slow ($O(n^2)$)	Fast ($O(n \log n)$)
Deterministic?	✓ Yes	✗ No	✗ No
Local structure	Partial	✓ Excellent	✓ Excellent
Global structure	✓ Yes	✗ No	Partial
Interpretable axes	✓ Yes	✗ No	✗ No
Explained variance	✓ Yes	✗ No	✗ No
Add new points	✓ Yes	✗ No	✓ Yes
Best for	Preprocessing	Visualization	Both

When to Use Which Method

- **Use PCA when:**
  - You need *interpretable* components ("what drives the variation?")
  - You want a *preprocessing* step before a classifier
  - Speed matters or dataset is very large
  - You need to explain how much information is retained
- **Use t-SNE when:**
  - You want to *visualize clusters* in 2D
  - Dataset is small–medium ($<$ 50K points)
  - You want the most detailed local structure
- **Use UMAP when:**
  - You want fast visualization for *large* datasets
  - Global relationships between clusters matter

The Practical Pipeline: PCA + t-SNE/UMAP

The standard workflow combines methods:



Why use PCA first?

1. **Denoise:** remove dimensions that are mostly noise
2. **Speed:** t-SNE on 50 features is much faster than on 300
3. **Better results:** t-SNE/UMAP work better on cleaner data

Best practice

`PCA(n_components=50)` → then `TSNE()` or `UMAP()`. This is what most practitioners do!

Complete Pipeline in Python


```python
import numpy as np
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
import umap
import matplotlib.pyplot as plt

# Load word embeddings: shape (n_words, 300)
X = load_word_vectors(words)

# Step 1: PCA to 50D (denoise + speed up)
pca = PCA(n_components=50)
X_50 = pca.fit_transform(X)
print(f"Variance kept: "
      f"{sum(pca.explained_variance_ratio_):.1%}")

# Step 2a: t-SNE to 2D
tsne = TSNE(perplexity=30, random_state=42)
X_tsne = tsne.fit_transform(X_50)

# Step 2b: UMAP to 2D
reducer = umap.UMAP(n_neighbors=15, min_dist=0.1)
X_umap = reducer.fit_transform(X_50)
```

Plotting Side by Side

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# PCA plot
axes[0].scatter(X_50[:, 0], X_50[:, 1],
                c=colors, s=20)
axes[0].set_title('PCA (first 2 PCs)')

# t-SNE plot
axes[1].scatter(X_tsne[:, 0], X_tsne[:, 1],
                c=colors, s=20)
axes[1].set_title('t-SNE (perplexity=30)')

# UMAP plot
axes[2].scatter(X_umap[:, 0], X_umap[:, 1],
                c=colors, s=20)
axes[2].set_title('UMAP (n_neighbors=15)')

# Annotate words
for i, word in enumerate(words):
    for ax in axes:
        ax.annotate(word, (X_method[i,0], X_method[i,1]),
                    fontsize=6)
plt.tight_layout()
plt.show()
```

Day 14: Key Takeaways

1. **Dimensionality reduction** compresses high-D data to low-D while keeping structure
2. **PCA** is linear: finds max-variance directions, fast, interpretable, good for preprocessing
3. **t-SNE** is nonlinear: preserves neighborhoods, great for visualization, slow
4. **UMAP** is nonlinear: fast like PCA, visual quality like t-SNE, preserves global structure
5. **Best practice:** PCA (denoise) $\rightarrow$ t-SNE/UMAP (visualize)

In your lab: apply all three methods to word embeddings and see animals, countries, and verbs form distinct clusters!

Next: Day 15 — CV Competition

Day 16: Clustering — "You saw the clusters. Now find them algorithmically."

Quick Reference Card

Method	Python Code	Key Parameter
PCA	`PCA(n_components=k).fit_transform(X)`	`n_components`
t-SNE	`TSNE(perplexity=30).fit_transform(X)`	`perplexity` (5–50)
UMAP	`UMAP(n_neighbors=15).fit_transform(X)`	`n_neighbors`, `min_dist`

Imports:

```
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
import umap   # pip install umap-learn
```

Typical pipeline:

```
X_50 = PCA(50).fit_transform(X_300)
X_2d = UMAP().fit_transform(X_50)
```

Credits

KAUST Academy

King Abdullah University of Science and Technology

Dimensionality reduction content partially adapted from

KAUST Academy Unsupervised Learning course materials

KAUST–Mawhiba IOAI 2026 Training Program